

POST MORTEM

Aplicação da Classe Pilha

Problema II - Torre de Hanói

MAE014 - Tópico em Ciência de Dados B: Estrutura de Dados

Integrantes	DRE
Alanna Figueiredo Simões	120053919
Artur de Melo Brito	117224234
Emanuel Cardoso	122033151
Felipe Rodrigues Schoeffer	123311188

Log de Iteração e Prompts

Prompt 1 - Código Bruto

No primeiro prompt foi anexado o arquivo pilhas_refatorado.ipynb, resultado do trabalho anterior, como base para utilização da classe Pilha. A partir dele foi dado o enunciado do trabalho que gerou o código bruto.

tomando como implementação da classe pilha o arquivo anexado: construa um programa que soluciona o problema da “Torre de Hanói” com N discos usando uma rotina recursiva. A Torre em questão é uma Pilha de N discos com raios crescentes [...]

```

def mostra_pinos(pino_inicial, pino_intermediario, pino_destino,
movimentos):
    print()
    print(f"Posição: {movimentos} movimentos")
    print()

    pinos = [
        pino_inicial,
        pino_intermediario,
        pino_destino
    ]

    nomes = [
        "Pino inicial",
        "Pino intermediário",
        "Pino destino"
    ]

    for nome, pino in zip(nomes, pinos):
        elementos = []

        for i in range(pino.tamanho()):
            elementos.append(str(pino.dados[i]))

        print(f"{nome}: [ {' '.join(elementos)} ]")

    print()
def mover_disco(origem, destino):
    disco = origem.desempilha()

    # Verifica se o movimento respeita a regra
    if not destino.pilha_esta_vazia():
        topo = destino.dados[destino.tamanho() - 1]

        if disco > topo:
            # Se o movimento for inválido, devolve o disco
            origem.empilha(disco)
            raise ValueError(
                "Não é permitido colocar um disco maior sobre um
menor."
            )

        destino.empilha(disco)

```

```

def hanoi(n, origem, destino, intermediario,
          movimentos, M, estado):
    if n == 0:
        return

    # Move os n-1 discos menores para o pino intermediário
    hanoi(
        n - 1,
        origem,
        intermediario,
        destino,
        movimentos,
        M,
        estado
    )

    # Move o maior disco restante para o destino
    mover_disco(origem, destino)

    estado[0] += 1

    # Mostra a situação a cada M movimentos
    if estado[0] % M == 0:
        mostra_pinos(
            origem,
            intermediario,
            destino,
            estado[0]
        )

        input("Pressione ENTER para continuar...")

    # Move os n-1 discos do intermediário para o destino
    hanoi(
        n - 1,
        intermediario,
        destino,
        origem,
        movimentos,
        M,
        estado
    )

```

```

# =====
# PROGRAMA PRINCIPAL
# =====

print("=====")
print("          TORRE DE HANÓI")
print("=====")

N = int(input("Digite a quantidade de discos N: "))

while N <= 0:
    print("N deve ser maior que zero.")
    N = int(input("Digite a quantidade de discos N: "))

M = int(input("Digite a quantidade de movimentos M entre visualizações:
"))

while M <= 0:
    print("M deve ser maior que zero.")
    M = int(input("Digite a quantidade de movimentos M entre
visualizações: "))

# Criação dos três pinos
pino_inicial = Pilha(N, int)
pino_intermediario = Pilha(N, int)
pino_destino = Pilha(N, int)

# Coloca os discos no pino inicial.
# N é o maior disco e 1 é o menor.
for disco in range(N, 0, -1):
    pino_inicial.empilha(disco)

# Apresenta a posição inicial
mostra_pinos(
    pino_inicial,
    pino_intermediario,
    pino_destino,
    0
)

```

```

input("Pressione ENTER para iniciar a solução...")

# Lista usada para manter o contador entre as chamadas recursivas.
# O valor começa em zero.
estado = [0]

# Resolve a Torre de Hanói
hanoi(
    N,
    pino_inicial,
    pino_destino,
    pino_intermediario,
    0,
    M,
    estado
)

# Caso o número total de movimentos não seja múltiplo de M,
# apresenta também a última posição.
if estado[0] % M != 0:
    mostra_pinos(
        pino_inicial,
        pino_intermediario,
        pino_destino,
        estado[0]
    )

print("=====")
print("          SOLUÇÃO FINAL")
print("=====")

mostra_pinos(
    pino_inicial,
    pino_intermediario,
    pino_destino,
    estado[0]
)

print(f"Total de movimentos: {estado[0]}")

# Quantidade teórica mínima de movimentos
print(f"Movimentos esperados: {2**N - 1}")

```